The Moth in the Machine

A History of Software Bugs and the Science of Testing

Brendan Shea, PhD

Programming and Problem Solving • Rochester Community and Technical College

Introduction

On September 9, 1947, engineers working on the Harvard Mark II computer found the machine behaving erratically. When they traced the problem to Relay #70, Panel F, they discovered the cause: a moth had flown into the relay and was preventing it from closing. They taped the moth into the logbook with the notation "First actual case of bug being found." Grace Hopper, the computer scientist leading the team, later popularized the story. The logbook still exists, moth and all, at the Smithsonian Institution.

Calling a malfunction a "bug" predates computers—Thomas Edison used the term for defects in his inventions in the 1870s—but the Harvard moth gave it a new life in computing, and the word has stuck ever since. Today a **software bug** means any flaw in a program that causes it to behave incorrectly.

What the moth story obscures is a deeper truth: software bugs are not incidental nuisances. They are a fundamental, irreducible feature of programming as a human activity. No matter how careful a programmer is, no matter how many tests are run, there is no method that can *prove* a non-trivial program is completely free of bugs. This is not pessimism—it is a mathematical and philosophical fact with profound implications for how we write, test, and reason about software.

Two Uncomfortable Truths About Bugs

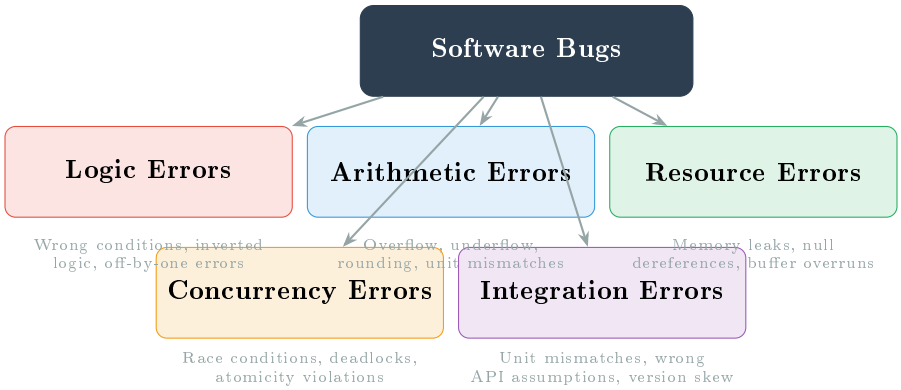
Truth 1: Every significant program that has ever been written contains bugs. Programs you use every day—your operating system, your web browser, your phone's apps—contain thousands of known bugs and an unknown number of unknown ones.

Truth 2: You can never prove a program is bug-free by testing it. You can only prove that bugs *exist* (by finding them), never that they *don't* exist.

These truths don't mean testing is pointless. They mean testing must be approached with humility and rigor—and that understanding *why* we can't prove absence of bugs illuminates *how* to test as effectively as possible.

1 A Taxonomy of Bugs

Not all bugs are alike. Understanding the categories helps both in recognizing them and in testing for them.



Each category has its own character—and its own testing challenges. Logic errors can often be found by carefully testing the boundary cases of conditions. Arithmetic errors require attention to data types and ranges. Concurrency errors may appear only under specific timing conditions and can be nearly impossible to reproduce reliably.

2 A History of Famous Bugs

The history of software bugs is also a history of the consequences of getting things wrong—from mildly annoying to catastrophic.

Mariner 1: The Missing Hyphen (1962)

NASA's Mariner 1 was designed to be the first spacecraft to fly past Venus. Shortly after launch in July 1962, the rocket began veering off course. The range safety officer destroyed the vehicle 293 seconds into flight at a cost of approximately $80 million (over $700 million in today's dollars).

The cause: a transcription error when converting a handwritten formula to computer code. A single overbar symbol (indicating a smoothed average) was omitted from the FORTRAN guidance program. Without it, the software used raw velocity data instead of filtered data, causing it to overcorrect for normal fluctuations.

Author Arthur C. Clarke called it "the most expensive hyphen in history." It is a canonical example of how a tiny, invisible error in code can have enormous real-world consequences.

Therac-25: When Bugs Kill (1985–1987)

The Therac-25 was a radiation therapy machine used to treat cancer. Between 1985 and 1987, it delivered massive radiation overdoses to at least six patients. Three died. The others suffered severe radiation burns.

The cause was a **race condition**—a class of concurrency bug in which the result of a computation depends on the precise timing of events. The Therac-25's control software had been adapted from an earlier model that had hardware safety interlocks. When those interlocks were removed to reduce cost, the software's timing-dependent safety checks were exposed. A trained operator who typed quickly could enter a sequence of commands that, by a matter of milliseconds, bypassed the safety checks and activated the high-power beam without the beam-shaping hardware in place.

The Therac-25 case is studied in every software engineering ethics course because it illustrates that software bugs are not merely technical problems—they have human consequences, and the engineers and institutions responsible for safety-critical software bear genuine moral responsibility.

Ariane 5: The Overflow Disaster (1996)

The European Space Agency's Ariane 5 rocket exploded 37 seconds after launch in June 1996, destroying four satellites worth approximately $370 million.

The cause: an **integer overflow**. A 64-bit floating-point value representing the rocket's horizontal velocity was converted to a 16-bit integer. The value was too large to fit—it exceeded the maximum a 16-bit integer can hold—and caused a hardware exception. The backup navigation computer, running identical software, failed in the same way. The primary flight computer, receiving garbage data, shut itself down. The rocket, receiving no guidance, self-destructed.

The irony: the overflowing variable tracked horizontal velocity during a ground alignment phase. On Ariane 5, this value was always large because the rocket launched from a different location than Ariane 4. The code had been reused from Ariane 4 without analyzing whether its assumptions still held. Reusing code without understanding its assumptions is a recurring theme in software disasters.

The Mars Climate Orbiter: Units Matter (1999)

The Mars Climate Orbiter was a $327 million NASA spacecraft designed to study the Martian climate. On September 23, 1999, it fired its engine to enter Mars orbit. The trajectory was slightly wrong, and the spacecraft either burned up in the atmosphere or skipped off it into space.

The cause: one engineering team was transmitting thruster force data in pound-force seconds; the receiving software expected newton-seconds. The two units differ by a factor of 4.45. Over the nine-month journey, the small error accumulated until the trajectory was fatally off.

This is not a programming bug in the narrow sense—the individual programs worked correctly. It is an *interface* bug: a mismatch in assumptions between components, undocumented and unchecked. Integration bugs of this type are among the hardest to catch because each component, tested in isolation, behaves correctly.

Knight Capital: 45 Minutes and $440 Million (2012)

Knight Capital Group was one of the largest US equity market makers. On August 1, 2012, a technician deployed new trading software to seven of eight production servers but forgot the eighth. The forgotten server was running old code that included a dormant feature called "Power Peg" that had been deactivated years earlier but never removed from the codebase.

When the market opened, Knight's system began executing thousands of unintended trades per second. The dormant code on the eighth server was activated by new message types and began buying high and selling low—the worst possible trading strategy—at machine speed. In 45 minutes, Knight lost $440 million. The company never recovered and was acquired months later.

The deployment failure was caused by a manual process with no automated verification that all servers had received identical code. A single absent checkbox in a release script ended a firm.

Boeing 737 MAX: Software and the Cost of Secrecy (2018–2019)

The Boeing 737 MAX was designed with larger engines mounted further forward than previous 737 models, which altered the aircraft's handling characteristics. To avoid expensive pilot retraining, Boeing developed the **MCAS** (Maneuvering Characteristics Augmentation System): software that automatically pushed the nose down if sensors detected a high angle of attack.

MCAS read from a single angle-of-attack sensor. If that sensor failed or gave erroneous readings, MCAS would push the nose down regardless of the pilot's inputs—and the published pilot training materials did not mention MCAS existed. On Lion Air flight 610 (October 2018) and Ethiopian Airlines flight 302 (March 2019), faulty sensor data triggered MCAS. Pilots unfamiliar with a system they didn't know existed could not stop it. 346 people died.

The technical failure was real: MCAS lacked sensor redundancy and failed-safe behavior. But the deeper failure was institutional: regulatory certification shortcuts, economic pressure, and the decision to conceal the system's existence from pilots who might otherwise have refused to fly without additional training.

3 The Philosophy of Testing: Popper and Dijkstra

3.1 Karl Popper and Falsificationism

In the 1930s, philosopher Karl Popper tackled a fundamental question: what separates science from non-science? His answer was **falsifiability**: a claim is scientific if and only if it is possible to specify an observation that would prove it *wrong*.

This might seem backwards. Shouldn't science be about proving things *right*? Popper's insight was that no number of confirming observations can *prove* a universal claim. You can observe a million white swans and conclude "all swans are white"—but one black swan refutes the claim forever. Confirmation accumulates; a single falsification destroys.

Karl Popper — *The Logic of Scientific Discovery* (1934)

"It must be possible for an empirical scientific system to be refuted by experience... I shall not require of a scientific system that it shall be capable of being singled out, once and for all, in a positive sense; but I shall require that its logical form shall be such that it can be singled out, in a negative sense: it must be possible for an empirical scientific system to be refuted by experience."

The asymmetry is fundamental: *confirmation is weak; falsification is strong.* A thousand passing tests suggest (but do not prove) correctness. One failing test proves a bug beyond doubt.

3.2 Dijkstra's Corollary

Edsger W. Dijkstra, one of the towering figures of computer science, applied this insight directly to software testing in 1969:

Edsger W. Dijkstra — "Notes on Structured Programming" (1969)

"Program testing can be used to show the presence of bugs, but never to show their absence!"

The exclamation mark is his. Dijkstra was not being dramatic—he was stating a logical consequence of the structure of software. A program with even modest complexity can be executed in a combinatorially enormous number of ways. A function that takes a 32-bit integer as input has over four billion possible inputs. Testing them all is generally impossible. No finite test suite can cover all possible executions of a non-trivial program.

Falsification (strong)

A test FAILS

Conclusion: the program has a bug.
(Certain. The bug is demonstrated.)

↓

Fix the bug.
Test again.

Corroboration (weak)

All tests PASS

Conclusion: no bug was found yet.
(Uncertain. Untested paths may hide bugs.)

↓

Write more tests.
Seek harder cases.

3.3 What This Means in Practice

The Popper–Dijkstra insight reshapes how a thoughtful programmer approaches testing:

- **Tests can never be "done."** Passing a test suite means you haven't found a bug in the tested paths—not that no bugs exist in untested paths. Seeking harder, more adversarial test cases is always worthwhile.
- **Each failing test is a gift.** A failing test reveals a real bug. The discomfort of a red test result is the system working as intended.
- **Test the boundaries.** Most bugs lurk at the edges—the off-by-one error, the empty list, the negative number, the maximum value, the null input. Testing typical inputs tells you the program works in typical cases; testing boundary inputs tells you it handles the exceptional cases that are most likely to hide bugs.
- **Tests as specifications.** A test suite doubles as an executable specification of what a program is supposed to do. A test that says `assertEquals(4, add(2, 2))` is both a check and a statement of intent.

Can We Ever Prove Correctness?

Dijkstra himself believed the answer was yes—but not through testing. He championed **formal verification**: using mathematical proof to establish that a program meets its specification for *all* possible inputs.
Formal methods are used in safety-critical domains: the Paris Métro Line 14 (driverless), aircraft autopilot systems, cryptographic protocols, and microprocessor designs. The **seL4 microkernel**—used in aerospace and defense systems—is formally verified: every line of code has been mathematically proven correct.

But formal verification is expensive and technically demanding. It requires specifying precisely what "correct" means, which is harder than it sounds. And a proof is only as good as the specification—if the specification itself is wrong, the proof is useless.
For most software, formal verification is impractical. For software where bugs can kill people, it may be the only responsible choice.

4 Unit Testing in Java with JUnit

4.1 What is a Unit Test?

A **unit test** is a piece of code that tests one small, specific piece of behavior (a "unit"—typically one method) in isolation from the rest of the program. Unit tests are:

- **Automated:** They run without human intervention and report pass or fail.
- **Fast:** Hundreds of unit tests should run in seconds.
- **Isolated:** Each test checks one thing; failures are easy to localize.
- **Repeatable:** Running them again always gives the same result.

JUnit is the standard unit testing framework for Java. It is included in most Java IDEs and build tools. A JUnit test is a regular Java class with methods marked with the `@Test` annotation.

4.2 A First Example: Testing a Buggy Method

Let's start with a deliberately flawed method—a function to compute the average of an array of integers—and see how tests reveal the bugs.

A Buggy average() Method — Can You Spot the Bugs?

```java
public class Statistics {

    // Returns the average of the values in the array.
    public static double average(int[] values) {
        int sum = 0;
        for (int i = 0; i <= values.length; i++) {    // BUG 1
            sum += values[i];
        }
        return sum / values.length;                   // BUG 2
    }
}
```

There are two bugs here. Can you find them before reading on?

- **Bug 1:** The loop condition is `i <= values.length` instead of `i < values.length`. This is an *off-by-one error*: on the last iteration, `values[values.length]` is accessed, which is beyond the end of the array. Java throws an `ArrayIndexOutOfBoundsException`.
- **Bug 2:** `sum / values.length` is *integer division* when both operands are `int`. The average of $\{1, 2\}$ would compute as `3 / 2 = 1`, not `1.5`.

Now let's write JUnit tests that expose both bugs:

JUnit Tests That Reveal the Bugs

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class StatisticsTest {

    // A normal case: should work for typical input
    @Test
    void averageOfThreeValues() {
        double result = Statistics.average(new int[]{2, 4, 6});
        assertEquals(4.0, result, 1e-9);
    }

    // Tests fractional result --- exposes Bug 2 (integer division)
    @Test
    void averageProducesFractionalResult() {
        double result = Statistics.average(new int[]{1, 2});
        assertEquals(1.5, result, 1e-9);  // FAILS: returns 1.0
    }

    // Single element --- also exposes Bug 1 via crash
    @Test
    void averageOfOneValue() {
        double result = Statistics.average(new int[]{42});
        assertEquals(42.0, result, 1e-9); // FAILS:
   ArrayIndexOutOfBoundsException
    }

    // Edge case: all the same value
    @Test
    void averageOfIdenticalValues() {
        double result = Statistics.average(new int[]{7, 7, 7, 7});
        assertEquals(7.0, result, 1e-9);
    }

    // Edge case: negative numbers
    @Test
    void averageWithNegatives() {
        double result = Statistics.average(new int[]{-3, -1, 1, 3});
        assertEquals(0.0, result, 1e-9);
    }

    // Edge case: what should happen with an empty array?
    @Test
    void averageOfEmptyArrayThrowsException() {
        assertThrows(IllegalArgumentException.class,
                     () -> Statistics.average(new int[]{}));
    }
}
```

Reading the assertEquals arguments

`assertEquals(expected, actual, delta)` takes three arguments:

1. **expected** — what you calculated the correct answer should be
2. **actual** — what the method actually returned
3. **delta** — tolerance for floating-point comparison (`1e-9` means they must agree to 9 decimal places)

Always put expected first and actual second. When a test fails, JUnit prints "expected ⟨X⟩ but was ⟨Y⟩"—this only makes sense if you follow the convention.

Now let's write the corrected method, informed by the failing tests:

The Corrected average() Method

```
public class Statistics {

    public static double average(int[] values) {
        if (values == null || values.length == 0) {
            throw new IllegalArgumentException(
                "Cannot compute average of empty array");
        }

        long sum = 0;                              // long avoids
    overflow
        for (int i = 0; i < values.length; i++) {  // < not <=
            sum += values[i];
        }
        return (double) sum / values.length;       // cast forces
    double division
    }
}
```

Running the same test suite against the fixed method: all six tests pass. Notice that the test for the empty array (which previously would have caused a `/ by zero` error) now passes because we explicitly handle that case.

4.3 The Ariane 5 Bug in Java

Let's recreate the integer overflow that destroyed the Ariane 5 rocket, this time in Java:

Recreating the Ariane 5 Overflow in Java

```
public class NavigationSystem {

    // Converts a velocity (as a double) to a 16-bit short for the
    legacy
    // navigation computer. DANGER: no range checking!
    public static short velocityToShort(double velocity) {
        return (short) velocity;  // Silent data corruption if out of
    range!
```

```
    }

    public static void main(String[] args) {
        // Ariane 4 scenario: horizontal velocity stays small
        double ariane4Velocity = 15000.0;
        System.out.println("Ariane 4 velocity: " + velocityToShort(
    ariane4Velocity));
        // Prints: 15000  (correct)

        // Ariane 5 scenario: much higher horizontal velocity due to
    launch site
        double ariane5Velocity = 37000.0;
        System.out.println("Ariane 5 velocity: " + velocityToShort(
    ariane5Velocity));
        // Prints: 1464  (WRONG! Short.MAX_VALUE is 32767; 37000
    overflows silently)
    }
}
```

Tests That Would Have Caught the Overflow

```
@Test
void velocityConversionInRange() {
    assertEquals((short)15000, NavigationSystem.velocityToShort
    (15000.0));
}

@Test
void velocityConversionThrowsOnOverflow() {
    // This test fails against the original code ---
    // it just silently returns a wrong value.
    // The fix requires adding a range check.
    assertThrows(ArithmeticException.class,
                 () -> NavigationSystem.velocityToShort(37000.0));
}
```

The second test reveals the gap: without the `assertThrows`, a test of `velocityToShort(37000.0)` would *pass* (it returns `1464` without complaint), giving false confidence.

Testing for Failure Is as Important as Testing for Success

Many beginning programmers write tests only for the "happy path"—the expected, normal inputs. But bugs most often hide in:

- **Boundary values:** the minimum and maximum legal inputs
- **Just outside boundaries:** one less than minimum; one more than maximum
- **Special values:** zero, negative numbers, `null`, empty strings or arrays
- **Error cases:** inputs that should cause an exception

A test that checks `divide(10, 2) == 5` is useful. A test that checks `divide(10, 0)` throws an exception is *more* useful—because division by zero is exactly the case that production

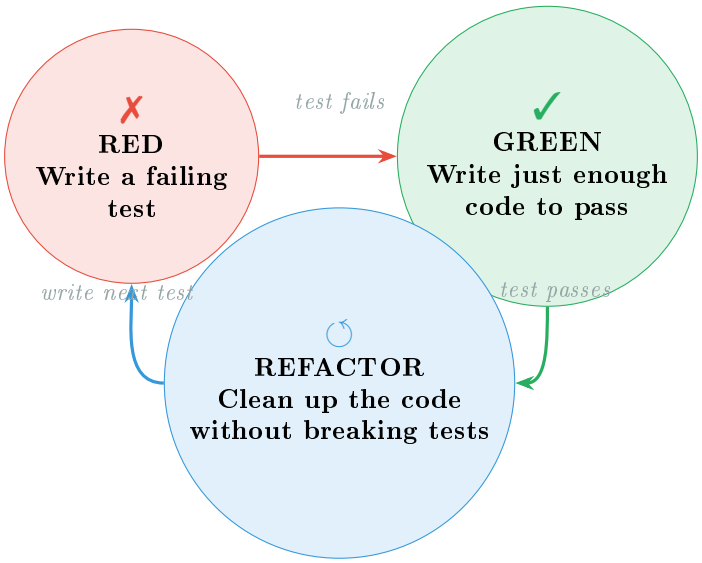
code is likely to receive and that a programmer might forget to handle.

5 Test-Driven Development

5.1 Reversing the Order

The natural impulse is to write code first and tests afterward. **Test-driven development** (TDD) reverses this: you write the test *before* you write the code it tests.

The TDD workflow follows a simple three-phase cycle:



5.2 TDD in Practice: Building a GradeBook

Let's build a simple `GradeBook` class using TDD. We start with no implementation—just tests that describe the behavior we want.

Step 1: Write Failing Tests First

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class GradeBookTest {
    private GradeBook book;

    @BeforeEach
    void setUp() {
        book = new GradeBook();  // Does not exist yet -- won't
   compile!
    }

    @Test
    void newGradeBookIsEmpty() {
        assertEquals(0, book.getCount());
```

```
    }

    @Test
    void addingGradeIncreasesCount() {
        book.addGrade(85.0);
        assertEquals(1, book.getCount());
    }

    @Test
    void averageOfOneGrade() {
        book.addGrade(90.0);
        assertEquals(90.0, book.getAverage(), 1e-9);
    }

    @Test
    void averageOfMultipleGrades() {
        book.addGrade(70.0);
        book.addGrade(80.0);
        book.addGrade(90.0);
        assertEquals(80.0, book.getAverage(), 1e-9);
    }

    @Test
    void highestGrade() {
        book.addGrade(72.0);
        book.addGrade(95.0);
        book.addGrade(88.0);
        assertEquals(95.0, book.getHighest(), 1e-9);
    }

    @Test
    void averageOfEmptyBookThrowsException() {
        assertThrows(IllegalStateException.class, () -> book.
   getAverage());
    }
}
```

Step 2: Write the Minimum Code to Make Tests Pass

```
import java.util.ArrayList;
import java.util.List;

public class GradeBook {
    private List<Double> grades = new ArrayList<>();

    public void addGrade(double grade) {
        grades.add(grade);
    }

    public int getCount() {
        return grades.size();
    }
```

```java
    public double getAverage() {
        if (grades.isEmpty()) {
            throw new IllegalStateException("No grades recorded");
        }
        double sum = 0;
        for (double g : grades) sum += g;
        return sum / grades.size();
    }

    public double getHighest() {
        if (grades.isEmpty()) {
            throw new IllegalStateException("No grades recorded");
        }
        double max = grades.get(0);
        for (double g : grades) {
            if (g > max) max = g;
        }
        return max;
    }
}
```

All six tests now pass. The tests specified the behavior; the implementation fulfilled it.

Why TDD Produces Better Code

TDD has practical benefits beyond correctness:

Forces you to think before you code. Writing a test requires knowing what the method *should* do—the input, the expected output, and the edge cases. This thinking often reveals ambiguities or design problems before any code is written.

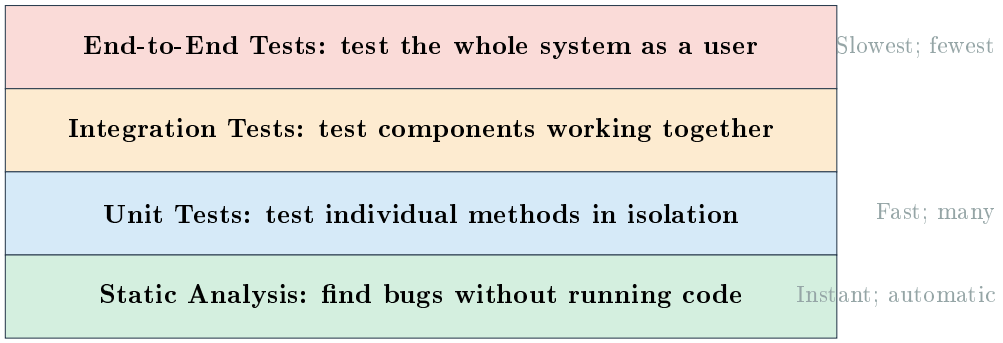
Produces naturally testable code. Code written test-first tends to have small methods with clear inputs and outputs—because that's what's easy to test. Code written implementation-first often has tangled dependencies that are difficult to test afterward.

Provides a safety net for changes. When you later modify the `GradeBook`—adding letter grades, weighting categories—the existing tests immediately tell you if you've broken existing behavior. This is called **regression testing**.

Documents intent. The test suite documents what the code is *supposed* to do, in executable form. New team members can read the tests to understand the expected behavior.

6 Beyond Unit Tests: A Testing Vocabulary

Unit tests are essential but not sufficient. A complete testing strategy layers several complementary approaches:



Integration tests verify that separately correct components work correctly *together*. The Mars Climate Orbiter failure was an integration bug: each team's software was correct in isolation. Integration tests would have verified that the units produced by one team were interpretable by the other.

End-to-end tests simulate real user scenarios through the complete system. They catch bugs that only emerge when all the pieces interact, but they are slower and more fragile than unit tests.

Static analysis tools examine source code without running it, looking for common bug patterns: unused variables, unreachable code, potential null dereferences, and type mismatches. Java's compiler itself performs basic static analysis. Tools like **SpotBugs** and **PMD** go further. Some languages (notably Rust) have static analysis so thorough that entire categories of bugs—null pointer errors, data races—are ruled out at compile time.

Property-based testing generates *random* inputs and tests that high-level properties hold for all of them—a powerful extension of the Popperian approach. Instead of writing specific test cases, you specify invariants: "the average should always be between the minimum and maximum," "sorting should never change the list's size." The framework then tries thousands of random inputs trying to find a counterexample.

7 The Human Side of Bugs

7.1 Blameless Postmortems

When a significant bug reaches production and causes harm, the instinctive response is to find the person who made the mistake and hold them accountable. This instinct is understandable but counterproductive.

Modern software engineering culture has increasingly adopted the **blameless postmortem**: a structured analysis of what went wrong that focuses on *systemic* causes rather than individual culpability. The reasoning is straightforward:

- Software is too complex for any individual to understand completely. Bugs are almost always the result of multiple contributing factors.
- Engineers who fear punishment for mistakes hide them or avoid taking on risky work. A culture of psychological safety produces better outcomes.
- Fixing the *system* that allowed the bug to exist prevents similar bugs in the future. Blaming an individual does nothing to fix the system.

Individual Responsibility vs. System Failure

The blameless postmortem approach has limits. Engineers on the Therac-25 ignored user reports of problems, dismissed error messages as display glitches, and deployed software they had not adequately tested for safety-critical use. The Boeing 737 MAX failures involved corporate decisions to conceal the MCAS system from pilots. In cases where individuals or organizations were reckless or dishonest, systemic framing can become a shield against deserved accountability.

A thoughtful position: most bugs emerge from systems, not malice or unique incompetence. But when individuals choose to cut corners, suppress safety concerns, or make decisions they know to be dangerous for financial gain, individual accountability is appropriate alongside systemic analysis. The question is not "blame or no blame" but "what causal story most accurately describes what happened, and what response would best prevent recurrence?"

7.2 Bug Reporting and Reproduction

A bug that cannot be reproduced cannot be fixed. When reporting a bug—to a colleague, to an open-source project, or in your own notes—the essential ingredients are:

1. **Steps to reproduce:** Precisely what actions or inputs trigger the bug.
2. **Expected behavior:** What the program *should* have done.
3. **Actual behavior:** What the program *did* instead, including any error messages.
4. **Environment:** Language version, operating system, relevant configuration.

A test that reliably reproduces the bug is the gold standard of a bug report. Before filing a bug report, write a unit test that fails because of the bug. When the bug is fixed, the test passes—and stays in the test suite to prevent regression.

Discussion Questions

Discussion Questions

1. **Popper and Testing:** Popper argued that no number of confirming observations proves a theory; one counterexample refutes it. How does this apply to software testing? If you run 10,000 tests and all pass, what have you learned? What haven't you learned? Does this make testing feel less valuable to you—or more?
2. **The Ariane 5 Lesson:** The Ariane 5 failure was caused by reusing code from Ariane 4 without analyzing whether its assumptions still held. Can you think of other contexts—outside of rocketry—where reusing something without checking its assumptions could cause serious problems?
3. **Therac-25 and Moral Responsibility:** The Therac-25 killed people because of a software bug. Who bears moral responsibility: the individual programmers who wrote the faulty code? The managers who removed hardware interlocks to reduce cost? The hospital administrators who purchased and deployed the machine? The regulators who approved it? How should responsibility be distributed in complex sociotechnical systems?

4. **TDD Trade-offs:** Test-driven development requires writing tests before writing implementation code. What would be the advantages of this approach in a student project? What would be the disadvantages? Under what circumstances might TDD be impractical?

5. **The Limits of Testing:** Dijkstra said testing can show the presence of bugs but never their absence. Does this mean we should pursue formal verification for all software? What would be the costs and benefits of requiring mathematically verified software for any system where lives are at stake?

Key Terms

Glossary of Key Terms

Blameless Postmortem	A structured analysis of a failure that focuses on systemic and process causes rather than individual blame, to enable learning and prevent recurrence.
Boundary Value	An input at the edge of a valid range (the minimum, the maximum, or just beyond either); bugs disproportionately hide at boundaries.
Edge Case	An input or condition at an extreme or unusual corner of a function's domain; often the last cases tested and the first to harbor bugs.
Falsifiability	Karl Popper's criterion for scientific claims: a claim is scientific only if it can in principle be refuted by observation; the asymmetry between confirmation and falsification.
Formal Verification	Mathematically proving that a program satisfies its specification for all possible inputs, as opposed to testing a finite sample of inputs.
Integer Overflow	A bug where an arithmetic result exceeds the maximum value that can be stored in the chosen integer type, producing a silently incorrect result.
JUnit	The standard Java unit testing framework; provides annotations (`@Test`) and assertion methods (`assertEquals`) for writing automated tests.
Off-by-One Error	A logic error where a loop or index is one position too far or too short; among the most common bugs in all of programming.
Race Condition	A concurrency bug where the outcome depends on the relative timing of events in different execution threads; can produce different results on different runs.
Regression Testing	Running an existing test suite after making changes to a program, to verify that previously working behavior has not been broken.
Software Bug	Any flaw in a program that causes it to produce incorrect behavior, crash, or otherwise deviate from its specification.

Static Analysis Examining source code without running it to find potential bugs, style violations, or security vulnerabilities.

Test-Driven Development A development practice in which tests are written before the implementation code they test, following a Red–Green–Refactor cycle.

Unit Test An automated test that checks one small unit of behavior (typically one method) in isolation from the rest of the system.

This case study is part of the Open Educational Resources for Programming and Problem Solving.
Licensed under Creative Commons Attribution 4.0 (CC BY 4.0).